A BEGINNER'S GUIDE, WRITTEN AROUND YOUR OWN CODE

React under the hood

What really happens when Tiko runs, what every function in **App.tsx** does, and how to organise a React project so it stays easy to understand.

How to use this guide
Read chapters 1–6 in order: each one builds on the previous. Keep **src/App.tsx** open next to the PDF and search (Ctrl+F) for the function names you see here. Chapters 7–9 are about your project specifically (login, the map of App.tsx, and how to restructure it).

Contents

What happens when you open localhost:5173 2

Fundamental #1 — a component is just a function 3

Fundamental #2 — props are a component's inputs 5

Fundamental #3 — state, and what a re-render really is 6

Lists, conditions and events 8

Effects, hooks and context 9

Routing and login under the hood 11

A map of every function in App.tsx 13

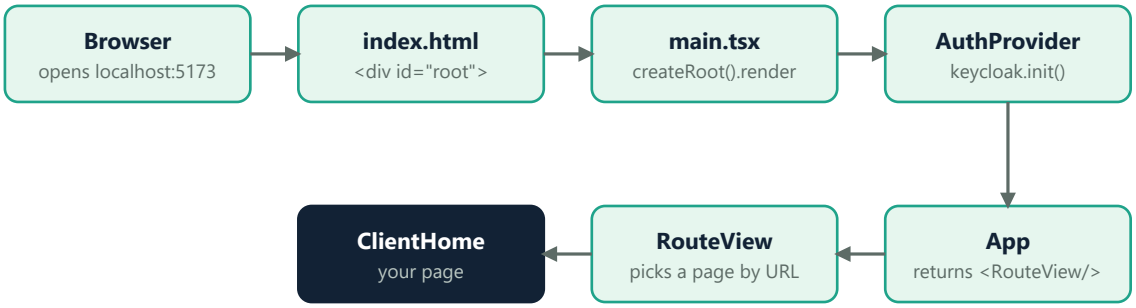
How to architect a React project 15

Glossary and what to learn next 18

CHAPTER 1

What happens when you open localhost:5173

Before learning any React rule, it helps to see the whole chain from “I typed a URL” to “I see a page”.



Step by step:

- **Vite** (started by `npm run dev`) is a small web server. Browsers can't read TypeScript or JSX, so Vite translates every `.tsx` file into plain JavaScript the moment the browser asks for it.
- The browser first downloads **index.html**. It's almost empty: just `<div id="root"></div>` and a `<script src="/src/main.tsx">`. Everything you see on screen is created by JavaScript afterwards.
- **main.tsx** runs `createRoot(document.getElementById("root")).render(<App/>)`. This hands the empty div to React and says “draw `<App/>` in here”.
- **AuthProvider** (in `auth.tsx`) wraps the app. It first talks to Keycloak to know if you are logged in, showing “Chargement…” meanwhile, then renders its children.
- **App** returns `<RouteView/>`, which looks at the URL (`/home`, `/profile`…) and picks one page component.
- That page returns a description of the UI (JSX). React turns that description into real HTML elements (the **DOM**).

> **The key idea**
> This is a **Single-Page Application (SPA)**. The server sends *one* HTML page, once. When you click a link inside Tiko, the browser does *not* download a new page: React just swaps which component is drawn inside the root div. That's why navigation feels instant.

CHAPTER 2

Fundamental #1 — a component is just a function

Every function in App.tsx whose name starts with a capital letter is a component: a function that returns a piece of UI.

Here is your **Brand** component (the "tiko." logo), re-formatted so it's readable:

src/App.tsx — Brand (simplified)

```
function Brand({ dark = false }: { dark?: boolean }) {
  return (
    <Link href="/" className="brand">
      <span className="brand-mark"><i/><i/><i/><i/></span>
      <span style={{ fontWeight: 800, color: dark ? '#f6faf8' : '#122235' }}>
        tiko<span style={{ color: '#1fa38a' }}>.</span>
      </span>
    </Link>
  );
}
```

Read it like this:

- `function Brand(...)` — a normal JavaScript function. The capital **B** is required: React treats `<brand>` as an HTML tag but `<Brand>` as your component.
- `{ dark = false }` — the inputs (called **props**, see chapter 3). `= false` is a default value.
- `: { dark?: boolean }` — this is TypeScript only. It describes the props' types; it disappears when the code runs.
- `return ( ... )` — the function returns **JSX**: HTML-looking syntax that describes what to show.
- Inside JSX, `{ }` means "evaluate this JavaScript". `dark ? '#f6faf8' : '#122235'` picks a colour.
- `className` instead of `class`, and `style={{ ... }}` takes a JavaScript object (outer braces = JS, inner = object).

What JSX really is

JSX is not HTML. Vite translates it into ordinary function calls. These two are identical:

JSX → JavaScript

```
// what you write
<Pill tone="amber">En attente</Pill>

// what the browser actually runs (after Vite compiles it)
jsx(Pill, { tone: "amber", children: "En attente" })
```

So `<Pill .../>` doesn't draw anything by itself. It creates a small object that *describes* "a Pill with these props". React later calls your `Pill` function with those props to find out what's inside. This tree of descriptions is often called the **virtual DOM**.

Using components inside components

You build big screens out of small components, like LEGO. `ClientHome` uses `ClientShell`, `EstablishmentCard`, `Button`, `Pill`… and `ClientShell` itself uses `Brand` and `Hub`. The result is a **tree** of components.

Rule: one root element
A component must return a single element. To return siblings without an extra div, wrap them in a **fragment** `<> ... </>` — you'll see this in `Hub`: `return <><button/>{open && <div/>}</>`.

CHAPTER 3

Fundamental #2 — props are a component's inputs

Props let the parent decide how a child looks. The child only reads them; it never changes them.

src/App.tsx — Pill

```
function Pill({ children, tone = 'green' }: {
  children: React.ReactNode;
  tone?: 'green' | 'amber' | 'red' | 'gray' | 'white';
}) {
  const colors = { green: ['#e6f6ee', '#258466'], amber: ['#fff2dc', '#ad6c12'], /* ... */ };
  return (
    <span className="status-pill"
          style={{ background: colors[tone][0], color: colors[tone][1] }}>
      {children}
    </span>
  );
}

// used elsewhere:
<Pill>Confirmé</Pill>                 // tone = 'green' (default)
<Pill tone="amber">En attente</Pill>  // tone = 'amber'
```

- The parent writes props like HTML attributes: `tone="amber"`. React collects them into one object and passes it as the function's single argument. `{ children, tone }` unpacks that object ("destructuring").
- `children` is special: it is whatever you put *between* the opening and closing tags (here, the text).
- The TypeScript type `'green' | 'amber' | ...` means only these exact strings are allowed — `<Pill tone="blue">` would be a red error in VS Code. That's TypeScript protecting you.

Layout components ("shells") use children

`ClientShell`, `TenantShell` and `ProviderShell` draw the header, navigation and menu, and put the page content in the middle via `children`. Every client page re-uses the same frame:

pattern used by all client pages

```
function ProfilePage() {
  return (
    <ClientShell active="Profil">       {/* header + nav, 'Profil' highlighted */}
      <SectionTitle eyebrow="VOTRE ESPACE" title="Profil" />
      ...the profile content...         {/* becomes ClientShell's children */}
    </ClientShell>
  );
}
```

Props flow down, events flow up
A parent passes data *down* through props. A child can't change its parent's data directly, but the parent can pass a *function* as a prop and the child calls it. Example: `<EstablishmentCard org={org} onClick={() => nav(`/establishment/${org.id}`)} />` — the card just calls `onClick`; the parent decides what that does.

CHAPTER 4

Fundamental #3 — state, and what a re-render really is

This is the single most important chapter. Once state and re-rendering "click", the rest of React becomes logical.

Props come from the parent. **State** is data a component owns and can change over time — is the menu open? which tab is selected? Your `Hub` component (the "Plan du prototype" button) is a perfect example:

src/App.tsx — Hub (simplified)

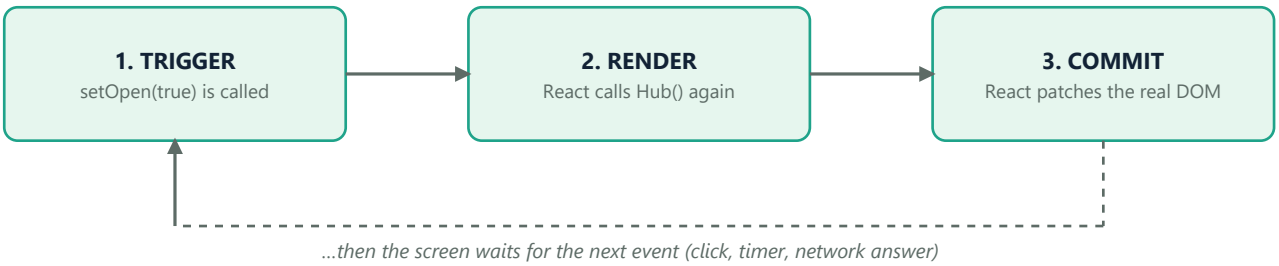
```
function Hub() {
  const [open, setOpen] = useState(false);     // (1) state: is the modal open?

  return (
    <>
      <button onClick={() => setOpen(true)}>    {/* (2) click → change state */}
        Plan du prototype
      </button>

      {open && (                                  // (3) only drawn when open is true
        <div className="overlay">
          <div className="hub-modal">
            <button onClick={() => setOpen(false)}>×</button>
            ...links...
          </div>
        </div>
      )}
    </>
  );
}
```

- **(1)** `useState(false)` asks React: "remember a value for me, starting at `false`". It gives back two things: the current value (`open`) and a function to change it (`setOpen`).
- **(2)** Clicking calls `setOpen(true)`.
- **(3)** `{open && (...)}` means "if `open` is true, show this; otherwise show nothing".

What happens, exactly, when you click



- **Trigger.** `setOpen(true)` does *not* change `open` immediately. It tells React: "the state of this Hub changed, please redraw it".
- **Render.** React calls the `Hub()` function *again, from the top*. This time `useState` returns `true`, so the function returns JSX that includes the modal.

- **Commit.** React compares the new JSX tree with the previous one (this comparison is called **reconciliation** or "diffing"). It sees that only the modal is new, so it inserts just that into the real page. The button is left untouched.

> **The mental model to keep**
> Your component is a **recipe**: *UI = f(state, props)*. You never say "add a div" or "hide this element". You change the state, and React re-runs the recipe and figures out the minimum changes to the screen.

Consequences that surprise beginners

- **Normal variables are forgotten** between renders, because the function runs again from scratch. `let count = 0` resets to 0 every render. Use `useState` for anything that must be remembered.
- **The state value doesn't change in the current run.** After `setOpen(true)`, `open` is still `false` until the next render.
- **Each component instance has its own state.** Every page draws its own `<Hub/>`, each with its own `open`.
- **When a component re-renders, its children re-render too** (they're called again). That's normal and usually fast.

Never modify state directly — make a new copy

React detects changes by checking if you gave it a *new* value. From your `Provider` page:

src/App.tsx — Provider

```
// NO  wrong: mutates the existing array, React doesn't notice
list.push(newOrganisation);

// OK  right (what your code does): build a NEW array
setList(l => [...l, { name: newOrg.name, kind: newOrg.type, plan: newOrg.plan, status: 'Actif' }]);

// OK  updating one item: map() returns a new array, {...o, status} a new object
setList(l => l.map(o => o.name === selected ? { ...o, status: 'Suspendu' } : o));
```

`l => ...` is the "updater" form: React passes you the latest value and you return the next one. `...` (spread) copies all items/fields into a new array/object.

CHAPTER 5

Lists, conditions and events

Three patterns you'll see on almost every line of App.tsx.

Showing a list with map()

src/App.tsx — ClientHome

```
const orgs = [
  { id: 'clinic', name: 'Clinique Al Amal', city: 'Casablanca', ... },
  { id: 'agency', name: 'Agence Guéliz',    city: 'Marrakech',  ... },
];

<div className="est-list">
  {orgs.map(org =>
    <EstablishmentCard key={org.id} org={org} onClick={() => nav(`/establishment/${org.id}`)} />
  )}
</div>
```

`map` turns an array of data into an array of JSX elements. The `key` prop must be unique and stable (an id, not the array index if the list can change): React uses it to know which card is which between renders.

Conditional rendering

Pattern	Meaning	Example in your code
`{cond && <X/>}`	Show X only if cond is true	`{open && <div className='overlay'>…}` (Hub)
`{cond ? <A/> : <B/>}`	Show A or B	`{tab==='Passés' ? <EmptyState/> : <Upcoming/>}` (Appointments)
`className={cond ? 'a' : ''}`	Change a style	`className={active===name ? 'client-nav active' : ''}`
`if (!x) return null`	Show nothing at all	`RequireAuth` in auth.tsx

Events

Event props (onClick, onChange, onSubmit…) take a **function** that React will call later:

event handlers

```
<button onClick={() => setOpen(true)}>      OK  a function: runs on click
<button onClick={setOpen(true)}>            NO  runs NOW, during render → infinite loop!
<button onClick={logout}>                   OK  passing an existing function is fine

<input value={phone} onChange={e => setPhone(e.target.value)} />   // a "controlled" input
<form onSubmit={e => { e.preventDefault(); ... }}>                  // stop the page reload
```

A **controlled input** keeps its text in state: every keystroke calls `setPhone`, React re-renders, and the input shows the new `value`. You'll see this pattern in the provider “Créer une organisation” wizard.

CHAPTER 6

Effects, hooks and context

Rendering must only compute UI. Anything that talks to the outside world goes in an effect.

useEffect — doing things after the screen updates

The render function should be "pure": same inputs → same JSX, no side effects. But apps must also start timers, call servers, redirect… That's what `useEffect` is for. It runs *after* React has updated the screen.

useEffect examples

```
// auth.tsx — AuthPage: if you're already logged in, go to /home
useEffect(() => {
  if (authenticated) nav('/home');
}, [authenticated, nav]);        // ← dependency array: re-run when these change

// a timer with cleanup (pattern from the old login page)
useEffect(() => {
  const id = setInterval(() => setTimer(t => t - 1), 1000);
  return () => clearInterval(id); // ← cleanup: runs before the next effect / when leaving the page
}, []);
```

Dependency array	When the effect runs
none: `useEffect(fn)`	after every render (rarely what you want)
empty: `useEffect(fn, [])`	once, when the component first appears ("mounts")
`useEffect(fn, [a, b])`	on mount, then again whenever a or b changed

Strict mode runs effects twice
In development React may mount components twice to catch bugs. That's why `auth.tsx` stores the `keycloak.init()` promise in a variable outside the component: Keycloak must only be initialised once.

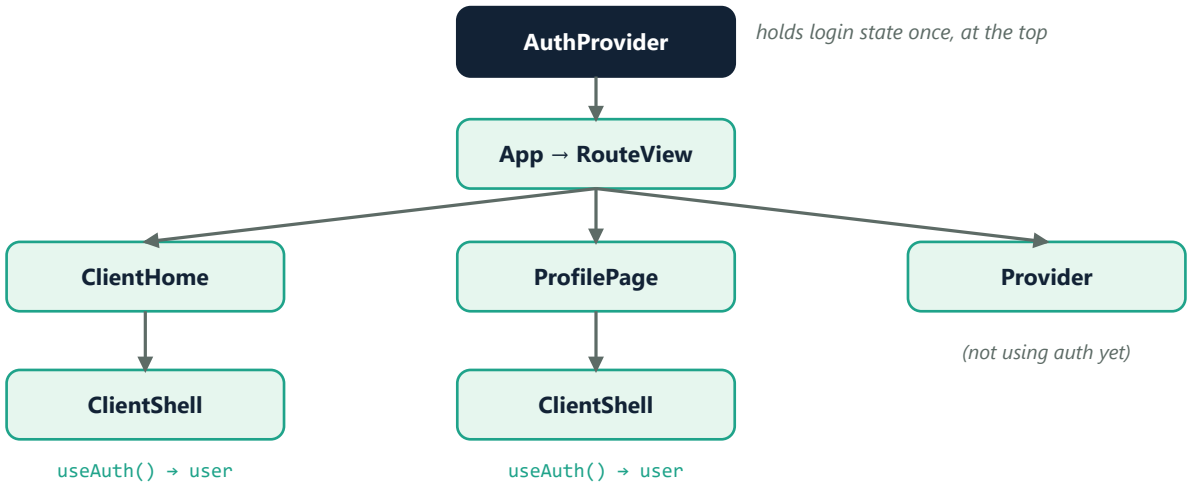
The rules of hooks

Functions starting with `use` (`useState`, `useEffect`, `useLocation`, `useAuth`) are **hooks**. Two rules:

- Call them only at the **top level** of a component (or of another hook) — never inside if, loops or event handlers.
- Call them in the **same order** every render. React identifies each piece of state by its position, not by name.

Context — sharing data without passing props everywhere

Many components need the logged-in user: the header (`ClientShell`), the greeting (`ClientHome`), the profile. Passing `user` as a prop through every level ("prop drilling") gets painful. **Context** lets a component high in the tree *provide* a value that any descendant can read directly.



src/auth.tsx

```
// auth.tsx (simplified)
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [authenticated, setAuthenticated] = useState(false);
  // ... keycloak.init() in a useEffect ...
  return <AuthContext.Provider value={{ authenticated, user, login, logout }}>
           {children}
         </AuthContext.Provider>;
}

export function useAuth() { return useContext(AuthContext); }   // a "custom hook"

// any component, anywhere below AuthProvider:
const { user, logout } = useAuth();
```

`useAuth` is a **custom hook**: just a function that uses other hooks. Custom hooks are how you move logic out of components so the component only describes UI.

CHAPTER 7

Routing and login under the hood

How the URL picks a page, and what keycloak-js does when you click “Se connecter”.

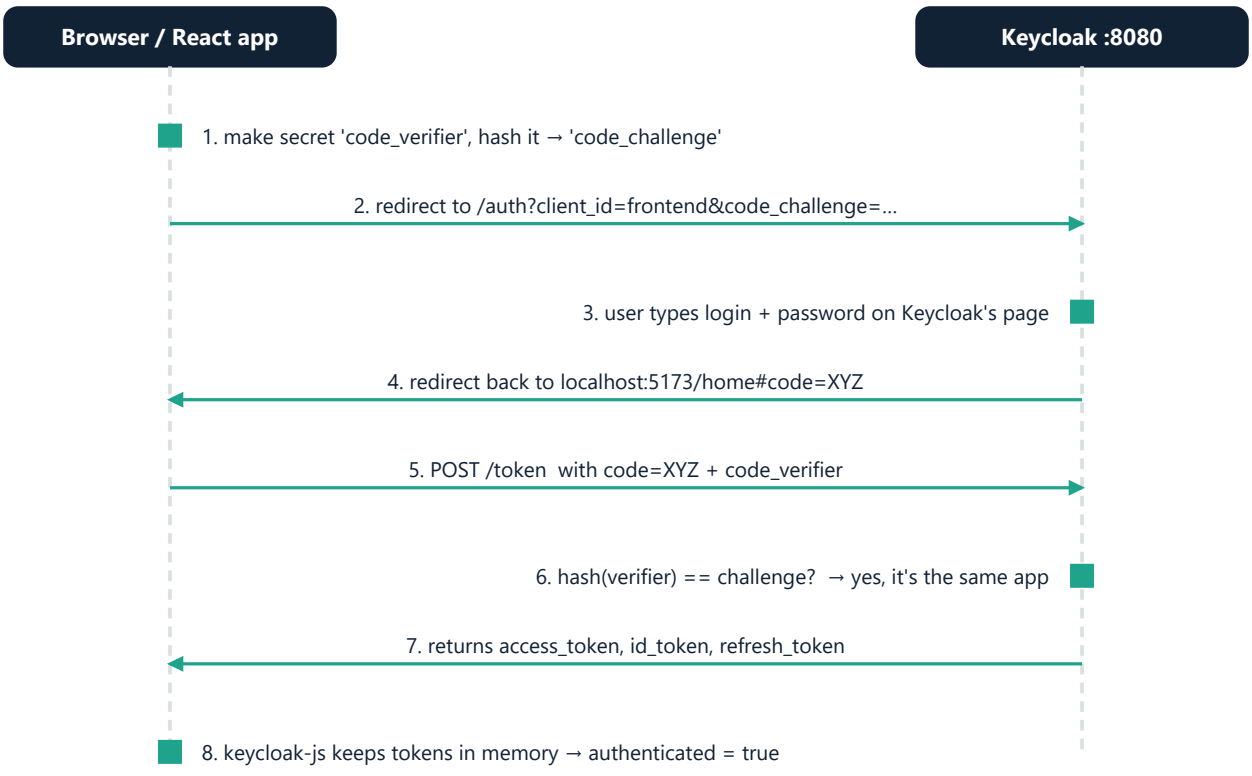
Routing with wouter

src/App.tsx — RouteView (simplified)

```
function RouteView() {
  return (
    <Switch>                                        {/* render the FIRST matching route */}
      <Route path="/" component={Landing} />
      <Route path="/login" component={AuthPage} />
      <Route path="/home" component={protect(ClientHome)} />
      <Route path="/establishment/:id">          {/* :id is a URL parameter */}
        {params => <RequireAuth><Establishment id={params.id} /></RequireAuth>}
      </Route>
      ...
      <Route>Cette page s'est éclipsée.</Route>    {/* no path = 404 fallback */}
    </Switch>
  );
}
```

- `<Link href="/search">` changes the URL *without* reloading (it uses the browser History API). wouter notices, `RouteView` re-renders and shows another page.
- `const [location, nav] = useLocation()` gives the current URL and a `nav("/home")` function to move in code.
- `protect(ClientHome)` wraps the page in `<RequireAuth>`: if you're not logged in, it calls `login()`, which sends you to Keycloak.

The login flow: Authorization Code + PKCE



- **Why PKCE?** A browser app can't keep a client secret (anyone can read its JavaScript). Instead, keycloak-js invents a random secret per login (`code_verifier`), sends only its hash at step 2, and reveals the original at step 5. Someone who steals the `code` from the URL can't use it without the verifier.
- **Where tokens live:** only in memory, inside the `keycloak` object. The cookies you saw (`KEYCLOAK_IDENTITY`…) are Keycloak's own session; they let Keycloak remember you, not your app.
- **check-sso:** on page load, `silent-check-sso.html` asks Keycloak in a hidden iframe "is this browser already logged in?", so a refresh doesn't force a new login.
- **Refresh:** access tokens expire quickly (minutes). `onTokenExpired` uses the refresh token to get a new one silently.
- **Calling your API:** `const token = await getValidToken()` then send `Authorization: Bearer <token>`. The backend verifies the token's signature with Keycloak's public key.

CHAPTER 8

A map of every function in App.tsx

App.tsx contains the whole prototype in one file. Here's what each piece is for, grouped by role.

Data (plain constants, not components)

Name	What it is
`orgs`	The two demo establishments (Clinique Al Amal, Agence Guéliz) with colours and addresses.
`providers`	Demo organisations listed in the provider console.
`tenantNavAdmin / tenantNavAgent`	Menu entries of the establishment back-office, for an admin vs an agent.
`iconsByNav, providerNav`	Icons and menu labels for the back-office and the provider console.

Small building blocks (reused everywhere)

Component	Props	What it draws
`Button`	kind, onClick, type	A button with one of 4 styles (dark, light, teal, plain).
`Brand`	dark	The “tiko.” logo, linking to the home page.
`Pill`	tone, children	A small coloured status label (“Confirmé”, “En attente”).
`Hub`	—	The floating “Plan du prototype” button + its navigation modal (has state).
`AvatarStack`	—	The overlapping avatars “SB MK YA +2k” on the landing page.
`SectionTitle`	eyebrow, title, sub	A page title with a small label above.
`EstablishmentCard`	org, onClick	One clickable establishment row (logo, city, distance, wait).
`ConfirmDialog`	title, message, onClose, onConfirm	A reusable “are you sure?” modal.
`StatTile`	title, value, sub, icon	A KPI tile in the back-office dashboard.
`SmoothChart`	title	The “Affluence du jour” chart.

Layout shells (frame around pages, via children)

Component	Used by
`ClientShell`	All customer pages: header with nav + logged-in user, mobile bottom bar.
`TenantShell`	The establishment back-office: sidebar, establishment switcher (clinic/agency).
`ProviderShell`	The provider (super-admin) console: dark sidebar and top bar.

Pages (one per route)

Component	URL	What it does
`Landing`	/	Public marketing page.
`AuthPage`	/login, /register	Left illustration + “Se connecter” button that redirects to Keycloak.
`ClientHome`	/home (login)	Greeting, current ticket banner, nearby establishments, next appointment.
`SearchPage`	/search (login)	Search and list establishments.
`Establishment`	/establishment/:id (login)	One establishment: choose a queue or a time slot.
`Success`	/success (login)	“C'est noté !” — ticket confirmed.
`TicketPage`	/tickets (login)	Live position in the queue.
`Appointments`	/appointments (login)	Upcoming/past appointments, cancel with ConfirmDialog.
`ProfilePage`	/profile (login)	User info from Keycloak, preferences, logout.
`TenantPage`	/tenant	Whole back-office: dashboard, counter, appointments, sites, queues, agents… (about 12 useState).
`TenantNamedPage`	/tenant/:page	Redirects /tenant/anything back to /tenant.
`ProviderLogin`	/provider	Mock super-admin login form (not wired to Keycloak yet).
`Provider`	/provider/*	Whole provider console: organisations, plans, health, support, audit log.
`Decisions`	/decisions	List of open product decisions.

Wiring

Name	Role
`protect(Page)`	Returns a version of Page wrapped in `<RequireAuth>` (cached so React sees the same component each render).
`RouteView`	The router: URL → page component.
`App`	The root component; currently just returns `<RouteView/>`.

> **(login) = requires login**
> Pages marked (login) redirect to Keycloak if you're not authenticated. The tenant and provider areas are still open.

CHAPTER 9

How to architect a React project

App.tsx works, but it is hard to read: one file, ~146 very long lines, 67 KB. Here's how professionals organise the same code, and a safe plan to get there.

What makes the current file hard

- **Everything is on a few giant lines.** A single line can hold a whole page. Your editor can't fold it, Git diffs are unreadable.
- **Data, UI and logic are mixed.** Demo data, styles, state and markup all live in the same function.
- **Huge components.** `TenantPage` and `Provider` each manage a dozen pieces of state and several screens at once.
- **One file for three products** (customer site, back-office, provider console) that have nothing in common.

A structure that scales: group by feature

suggested folder structure

```
src/
├── main.tsx                 entry point (createRoot + providers)
├── App.tsx                  only the routes
├── auth/
│   ├── keycloak.ts          the Keycloak instance
│   ├── AuthProvider.tsx     context + useAuth + RequireAuth
├── components/              small reusable pieces, no business logic
│   ├── Button.tsx  Pill.tsx  Brand.tsx  ConfirmDialog.tsx  SectionTitle.tsx
├── features/
│   ├── client/              customer site
│   │   ├── ClientShell.tsx
│   │   ├── pages/  HomePage.tsx  SearchPage.tsx  ProfilePage.tsx  ...
│   │   └── components/  EstablishmentCard.tsx
│   ├── tenant/              establishment back-office
│   │   ├── TenantShell.tsx
│   │   └── pages/  DashboardPage.tsx  CounterPage.tsx  AgentsPage.tsx ...
│   └── provider/            provider console
│       ├── ProviderShell.tsx
│       └── pages/  OrganisationsPage.tsx  PlansPage.tsx  AuditPage.tsx ...
├── api/
│   └── client.ts            fetch wrapper that adds the Bearer token
├── data/
│   └── demo.ts              orgs, providers (until the real API exists)
└── styles/                  tiko.css, workspace.css
```

Rules of thumb

Rule	Why / how
One component per file, named like the file	`Pill.tsx` exports `Pill`. Easy to find, easy to import.
Keep components small	If a component is longer than a screen or has 5+ useState, split it into sub-components.
One page = one route	Instead of `Provider` switching on a `page` state, give each screen its own URL (/provider/offres) and its own component. Back/forward buttons then work too.
Put state where it's used	Keep state in the lowest component that needs it. Lift it up to a common parent only when two siblings need it. Use context only for truly global data (user, theme).
Logic in hooks, UI in components	e.g. `useQueue(orgId)` fetches and updates the queue; the page just displays it.
All server calls in api/	One place that adds the token, handles errors and base URL. Components never call fetch directly.
Format automatically	Install Prettier (VS Code extension, format on save). It will turn the one-line components into readable code instantly.
Let TypeScript help	Define types once (`type Organisation = {...}`) and reuse them. Fix red squiggles instead of ignoring them.

Example: an API helper using the Keycloak token

a typical API layer

```
// src/api/client.ts
import keycloak from '../auth/keycloak';

export async function api<T>(path: string, options: RequestInit = {}): Promise<T> {
  await keycloak.updateToken(30);                    // refresh if it expires in < 30 s
  const res = await fetch(import.meta.env.VITE_API_URL + path, {
    ...options,
    headers: { ...options.headers,
               Authorization: `Bearer ${keycloak.token}`,
               'Content-Type': 'application/json' },
  });
  if (!res.ok) throw new Error(`${res.status} ${res.statusText}`);
  return res.json();
}

// in a page:
const [tickets, setTickets] = useState<Ticket[]>([]);
useEffect(() => { api<Ticket[]>('/tickets').then(setTickets); }, []);
```

A safe migration plan (one small step at a time)

Step	Do this	Check
1	Install Prettier and format App.tsx. Nothing changes in behaviour, everything becomes readable.	App looks identical
2	Move `orgs` and `providers` to `src/data/demo.ts` and import them.	`npm run typecheck`
3	Move the small blocks (Button, Pill, Brand, SectionTitle, ConfirmDialog) into `src/components/`.	typecheck + click around
4	Move the client shell and client pages into `features/client/`.	log in, visit every page
5	Do the same for tenant and provider. Then split TenantPage / Provider into one page per menu item with its own route.	every menu entry works
6	When the backend exists, replace demo data with `api()` calls, page by page.	network tab shows Bearer token

> **Tip**
> Do one step, run the app, and commit (git) before the next. If something breaks, you know exactly which small change caused it.

CHAPTER 10

Glossary and what to learn next

Term	In one sentence
Component	A function (capitalised) that returns JSX describing part of the UI.
JSX	HTML-like syntax that compiles to function calls creating UI descriptions.
Props	Read-only inputs a parent passes to a child component.
children	The special prop containing whatever is written between a component's tags.
State	Data a component remembers between renders; changing it triggers a re-render.
Render	React calling your component function to get the latest JSX.
Commit	React applying the differences to the real DOM.
DOM	The browser's live tree of HTML elements that you see on screen.
Reconciliation	React's comparison of the old and new JSX trees to find the minimal changes.
Hook	A function starting with “use” that plugs into React features (state, effects, context).
Effect	Code that runs after render to sync with the outside world (timers, network, redirects).
Context	A way to share a value with all descendants without passing props manually.
SPA	An app that loads one HTML page and swaps components when the URL changes.
Vite	The dev server/bundler that compiles TSX and serves your app.
Access token	Short-lived JWT proving who the user is; sent to APIs as a Bearer token.
PKCE	A proof (verifier + hashed challenge) that the app finishing login is the one that started it.

Learn next, in this order

- **react.dev** → **Learn** (official tutorial). Read “Describing the UI”, “Adding Interactivity” and “Managing State”. Now that you know the vocabulary, it will read quickly.
- **“You Might Not Need an Effect”** on react.dev — the most common beginner mistake is using useEffect where it isn't needed.
- Practice: take one page (e.g. ProfilePage), format it, move it to its own file, and explain each line to yourself using this guide.
- Then: data fetching with TanStack Query, forms with React Hook Form, and testing with Vitest + Testing Library.